

Week 3: Temporal Sequence Modeling – Giving the System Memory with LSTMs

1. Introduction and Objectives for Week 3

As Stage 1 of our data pipeline is finished, the system can now handle live video feeds, extract 1,662 3D spatial landmarks per frame using MediaPipe, and normalize this data to be scale-invariant. Nevertheless, a set of coordinates in one frame merely informs us of the physical location of a signer at a frozen microsecond time. The main goal of Week 3 was to start Stage 2 of the architecture: Temporal Sequence Modeling. The system needs to learn the direction, velocity, and transformation of gestures with time to learn American Sign Language (ASL). I set myself the objective of this week to shift away, through Computer Vision, to Deep Learning by building and training an instance of a Long Short-Term Memory (LSTM) neural network that can then handle our sequential spatial information.

2. The Data Structuring Challenge: From Frames to Sequences

The neural network architecture required the reorganization of the normalized spatial data of Week 2 before being written. A standard machine learning classifier takes one input to generate one output. In the case of temporal modeling, the network needs to have a sliding window set of inputs. I developed a data-loading script that would bundle the frame arrays individually in consecutive batches. I specified a fixed length of the sequence to the model, e.g. 30 frames, which corresponds to a duration of approximately one second of video at 30 FPS. Our neural network has now the input shape of a 3D tensor: (batch size, sequence length, number of features). This would be implemented by feeding the model batches of 30-frame sequences, with each frame having the 1,662 normalized spatial coordinates.

3. Handling Variable Gesture Lengths

One of the major challenges I met during the week was the inherent difference in sign duration. The WLASW dataset shows that a simple sign such as saying Hello can only require 15 frames to be performed whereas a complex sign could require 45 frames.

To input these variable-length videos into a neural network that requires a fixed sequence length, I provided padding and masking methods. In case a video is less than the assigned sequence length the end of the array is padded with zeros by the script. At the same time, a masking layer is also inserted into the neural network to specifically tell the neural network to disregard these zero-padded frames during training to ensure that the artificial data does not distort the model's perception of the natural resting state of the gesture.

4. Architecting the LSTM Network

The sequential formatted data was then used to build the fundamental deep learning framework using TensorFlow/Keras. I chose an LSTM (Long Short-Term Memory) network in preference to an ordinary RNN (Recurrent Neural Network). The traditional RNNs have the problem of vanishing gradient, i.e. forgetting the initial frames of a long video sequence. The LSTMs have internal gate mechanisms (forget, input, and output gates) which control the flow of information which enables them to store important spatial information at the start of a sign right up to the end.

The existing architecture is comprised of:

- An Input Layer that takes the 3D sequence tensors.
- Two Stacked LSTM Layers of 64 and 128 units each. Layering up the layers enables the network to acquire more complex and highlevel temporal representations of the hand and body trajectories.
- Dropout Layers (Regularization) with a rate of 0.4 to randomly shut down neurons during training. This will avoid overfitting the model to signers in the WLASK dataset, and will generalize well to new, unseen users.
- a dense Output Layers to decode the learned temporal patterns and produce a probability distribution over the ASL vocabulary of interest.

5. Initial Training and Validation

To test the architecture initially, and to train it on isolated signs, I initially ran trials on the WLASL dataset using isolated signs only, rather than continuous complex sentences.

I trained the model with Adam optimizer and categorical cross-entropy loss and 200 epochs. The outcomes were very promising. With the help of the TensorBoard, I was able to check the loss curves, thus ensuring that the LSTM was indeed learning the temporal dependencies. It may distinguish between signs that appear the same in a fixed picture but have different movement paths (as with the signs of "Please" and "Sorry" which both involve a flat hand on the chest but make use of different circular movements).